

Project Internet of Thing.

Smart Desk Assistant

สาเหตุที่นำ IoT มาช่วย

โปรเจกต์นี้มุ่งเน้นแก้ไข คือพฤติกรรมการทำงานที่ไม่ถูกสุขลักษณะซึ่งนำไปสู่ Office Syndrome และ สายตาสั้น/ล้า การนำเทคโนโลยี IoT มาใช้จึงเป็นการเปลี่ยนจากการเตือนด้วยตัวเองมาเป็น ระบบอัตโนมัติ (Automation) ที่มีประสิทธิภาพสูงกว่า ดังนี้:

- **Real-time Monitoring:** เพื่อให้สามารถตรวจจับพฤติกรรม ได้ตลอดเวลาแบบวินาทีต่อวินาที ซึ่งคนส่วนใหญ่ไม่สามารถทำได้ขณะโฟกัสกับงาน
- **Data-Driven Decisions:** นำข้อมูลจากเซนเซอร์ มาประมวลผลเพื่อปรับเปลี่ยนระยะเวลาการทำงาน ให้เหมาะสมกับสภาพร่างกายและสิ่งแวดล้อมของแต่ละบุคคลจริง
- **Interactive Feedback:** ใช้ Two-way Communication ระหว่างอุปกรณ์และ Cloud เพื่อแจ้งเตือนผ่านช่องทางที่ผู้ใช้เห็นได้ง่าย เช่น Telegram หรือ Dashboard

ปัญหาที่ต้องการแก้ไข

01

พฤติกรรมการนั่งไขว่ห้างเกินไป: ผู้ใช้งานมักโน้มตัวเข้าหาหน้าจอโดยไม่รู้ตัวเมื่อมีสมาธิสูง

02

สภาพแวดล้อมที่ไม่เหมาะสม: ความสว่างของแสงไฟ อุณหภูมิที่ร้อนเกินไป ซึ่งส่งผลต่อประสิทธิภาพการทำงานและสุขภาพตา

03

การทำงานต่อเนื่องนานเกินไป: ผู้ใช้ขาดระบบการจัดการเวลาที่ยืดหยุ่น



อุปกรณ์ที่ใช้งานในระบบ

หน่วยประมวลผลหลัก

ESP32: ทำหน้าที่เป็นสมองกลหลักของระบบ รองรับการเชื่อมต่อ Wi-Fi และ Bluetooth เพื่อส่งข้อมูลขึ้นระบบ Cloud และประมวลผล Logic ต่างๆ

เซนเซอร์ตรวจวัด

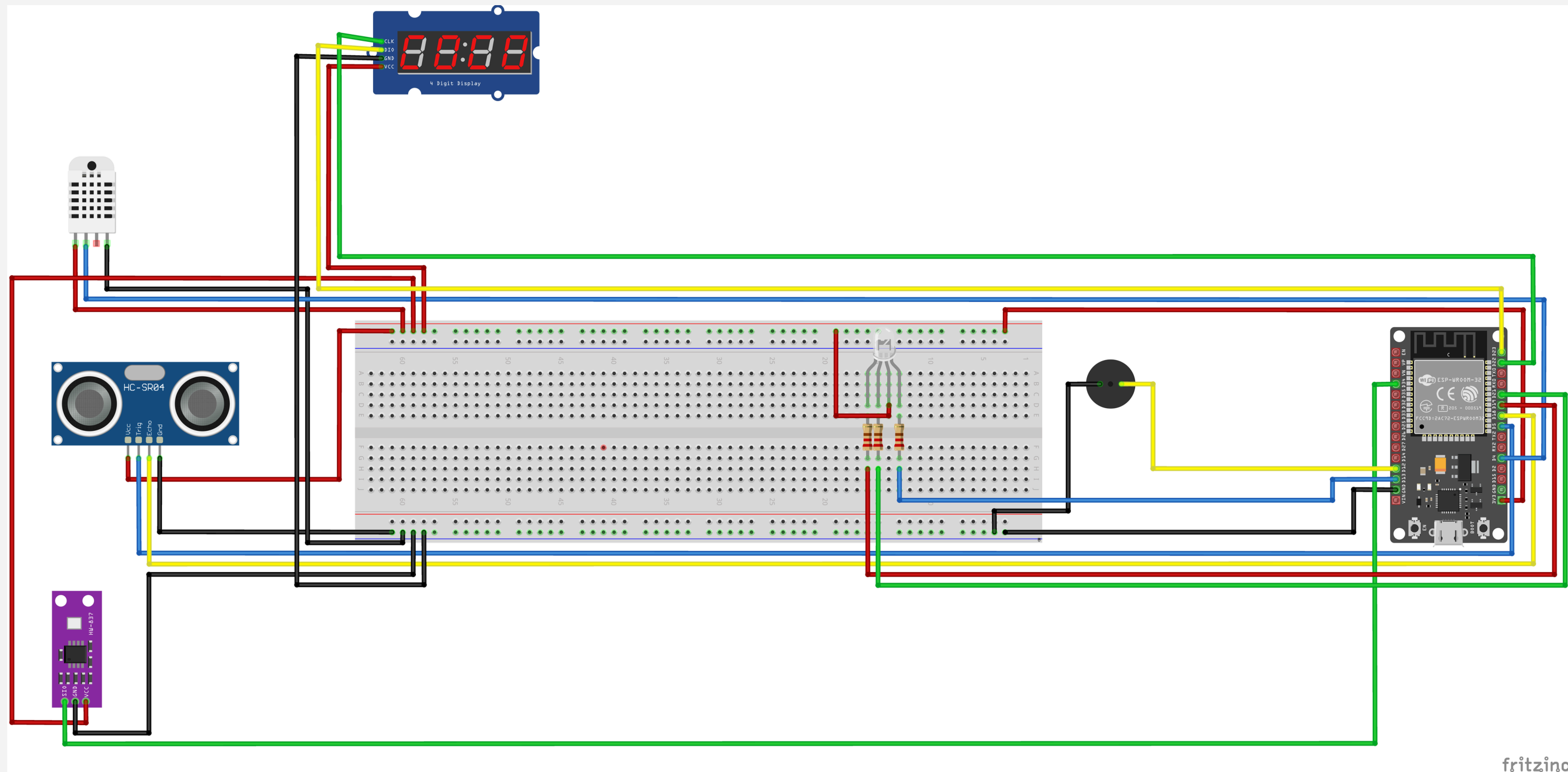
- **Ultrasonic Sensor:** วัดระยะห่างระหว่างใบหน้าของผู้ใช้งานกับหน้าจอคอมพิวเตอร์
- **LDR (Light Sensor):** วัดความสว่างของแสงในบริเวณโต๊ะทำงาน
- **DHT22:** วัดอุณหภูมิและความชื้นในอากาศ

อุปกรณ์แสดงผลและแจ้งเตือน

- **TM1637 (7-Segment Display):** หน้าจอตัวเลขสำหรับสลับการแสดงผลระหว่าง
- **RGB LED:** ไฟแสดงสถานะสุขภาพตามระบบคะแนน (สีเขียว = ปกติ, สีเหลือง = เริ่มหักคะแนน, สีแดง = วิกฤต)
- **Piezo Buzzer:** อุปกรณ์ส่งเสียงเตือนเบาๆ เมื่อระบบตรวจพบพฤติกรรมที่ไม่เหมาะสม

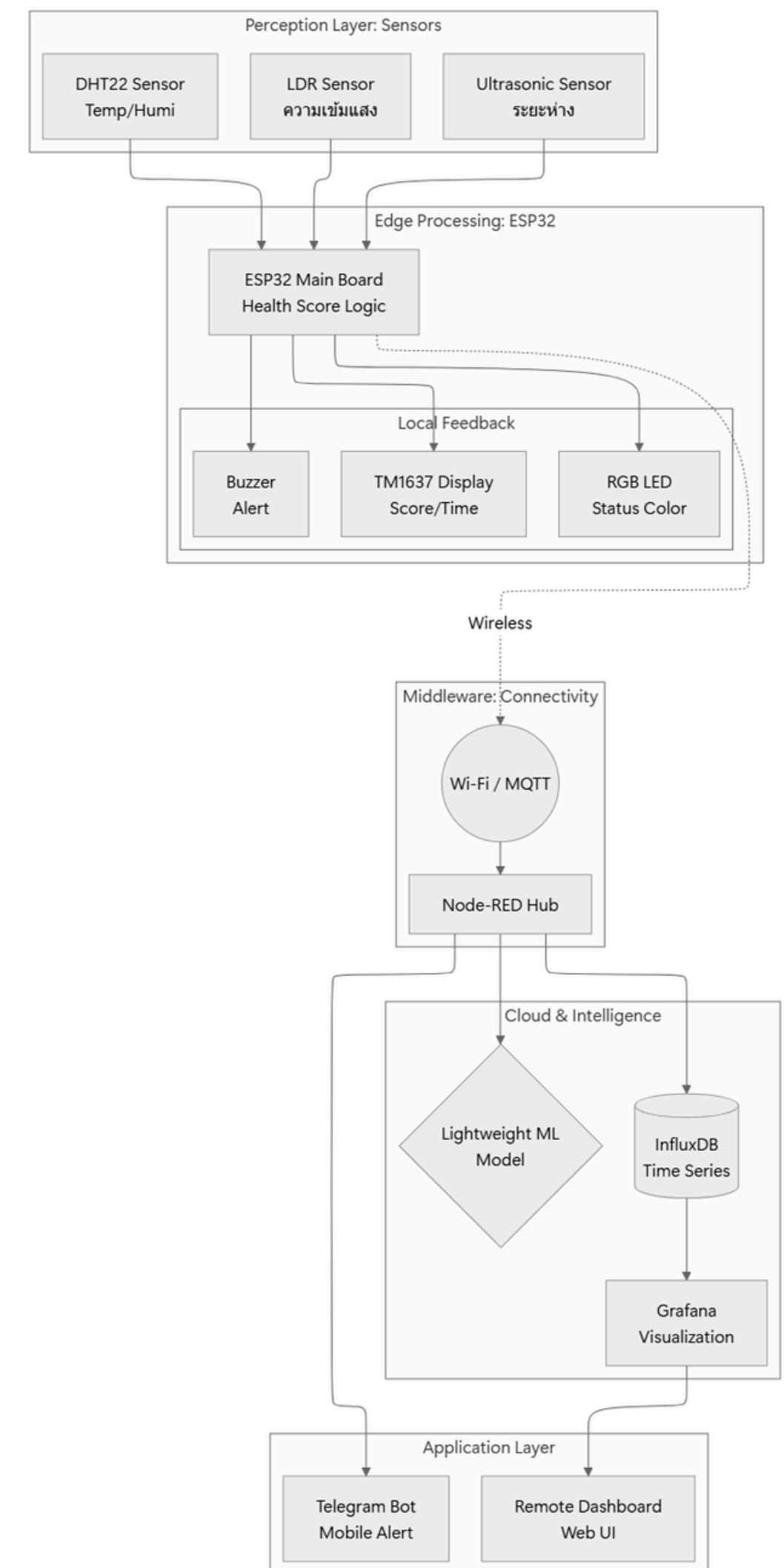


รูปวงจรที่ถูกต้อง Fritzing

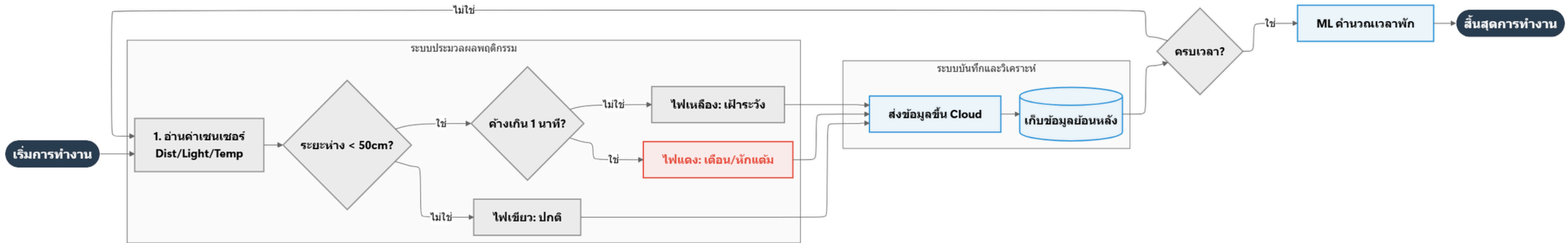


สถาปัตยกรรมระบบ

- **Perception Layer:** ส่วนรับข้อมูลจากสภาพแวดล้อมจริง (ระยะห่าง, แสง, อุณหภูมิ)
- **Edge Processing (ESP32):** ทำหน้าที่เป็น "สมองส่วนหน้า" ประมวลผล Logic สำคัญและแจ้งเตือนทันที (Local UI) เพื่อความรวดเร็วและเสถียรแม้ไม่มีอินเทอร์เน็ต
- **Node-RED:** ทำหน้าที่เป็นตัวกลาง (Central Hub) ในการกระจายข้อมูลจาก Hardware ไปยัง Service ต่างๆ บน Cloud
- **Cloud & Intelligence:** ส่วนจัดเก็บข้อมูลระยะยาวใน InfluxDB เพื่อนำไปวิเคราะห์ด้วย ML และแสดงผลผ่าน Grafana
- **Application Layer:** ส่วนที่ผู้ใช้โต้ตอบ (Remote UI) ทั้งการดูผลสทิตีย้อนหลังและการรับแจ้งเตือนผ่าน Telegram



Flowchart



1. การเริ่มต้นและรับข้อมูล

- **Start & Setup:** เมื่อเปิดเครื่อง ESP32 จะทำการเชื่อมต่อ Wi-Fi และโปรโตคอล MQTT เพื่อเตรียมส่งข้อมูลเข้าสู่ระบบ Cloud
- **Sensor Reading:** ระบบจะอ่านค่าจากเซนเซอร์ 3 ชนิดพร้อมกัน ได้แก่ (Ultrasonic), (LDR) (DHT22) ทุกๆ 5 วินาที

2. การประมวลผลพฤติกรรมอัจฉริยะ ระบบเพื่อแก้ปัญหาพฤติกรรมที่ไม่น่าเหมาะสม:

- **Distance Check:** ตรวจสอบว่าผู้ใช้นั่งใกล้จอเกินกว่า 50 เซนติเมตร หรือไม่
- **The 1-Minute Rule:** หากนั่งใกล้เกินไป ระบบจะยังไม่เตือนทันทีแต่จะเริ่มนับเวลาถอยหลัง 1 นาที
 - **หากขยับตัวออก:** ระบบจะ Reset กลับไปเป็นสถานะปกติ (ไฟสีเขียว)
 - **หากยังนั่งใกล้ค้างไว้:** เมื่อครบ 1 นาที ระบบจะส่งสัญญาณเตือน และทำการ หักคะแนนสุขภาพ

3. การเชื่อมต่อ Cloud และการจัดเก็บข้อมูล

- **Data Sync:** ข้อมูลพฤติกรรมและค่าเซนเซอร์ทั้งหมดจะถูกแพ็คและส่งผ่าน MQTT ไปยัง Node-RED
- **Storage:** ข้อมูลจะถูกบันทึกลงใน InfluxDB ซึ่งเป็น เพื่อใช้ในการดูสถิติย้อนหลังและวิเคราะห์แนวโน้มสุขภาพผ่าน Dashboard

4. การจัดการรอบการทำงานและ ML

- **Session Timer:** ระบบกำหนดรอบการทำงานไว้ที่ 30 นาที
- **ML-Lite Calculation:** เมื่อครบกำหนดเวลา ระบบจะไม่สั่งให้พักแบบคงที่ แต่จะนำ "คะแนนสุขภาพสะสม" ในรอบนั้นมาคำนวณด้วยอัลกอริทึม ML เพื่อกำหนดว่า:
 - **ถ้าพฤติกรรมดี (คะแนนสูง):** รอบถัดไปอาจได้ทำงานนานขึ้น
 - **ถ้าพฤติกรรมแย่ (คะแนนต่ำ):** ระบบจะบังคับให้พักนานขึ้น และลดเวลาการทำงานรอบถัดไปลง

การติดตั้งและตั้งค่า Node-RED, Time Series DB

MQTT Topic



(ESP32 Subscribe)

smartdesk/control

(ESP32 Publish)

smartdesk/telemetry

smartdesk/events

Node Red Install



node-red-dashboard

node-red-contrib-blynk-iot

การติดตั้งและตั้งค่า Node-RED, Time Series DB

MQTT Node

- MQTT In: รับค่าจาก smartdesk/telemetry และ smartdesk/events
- MQTT Out: ส่งคำสั่งไปที่ smartdesk/control
- Server Config: เชื่อมที่ Broker local (192.168.100.17:1883)

การติดตั้งและตั้งค่า Node-RED, Time Series DB

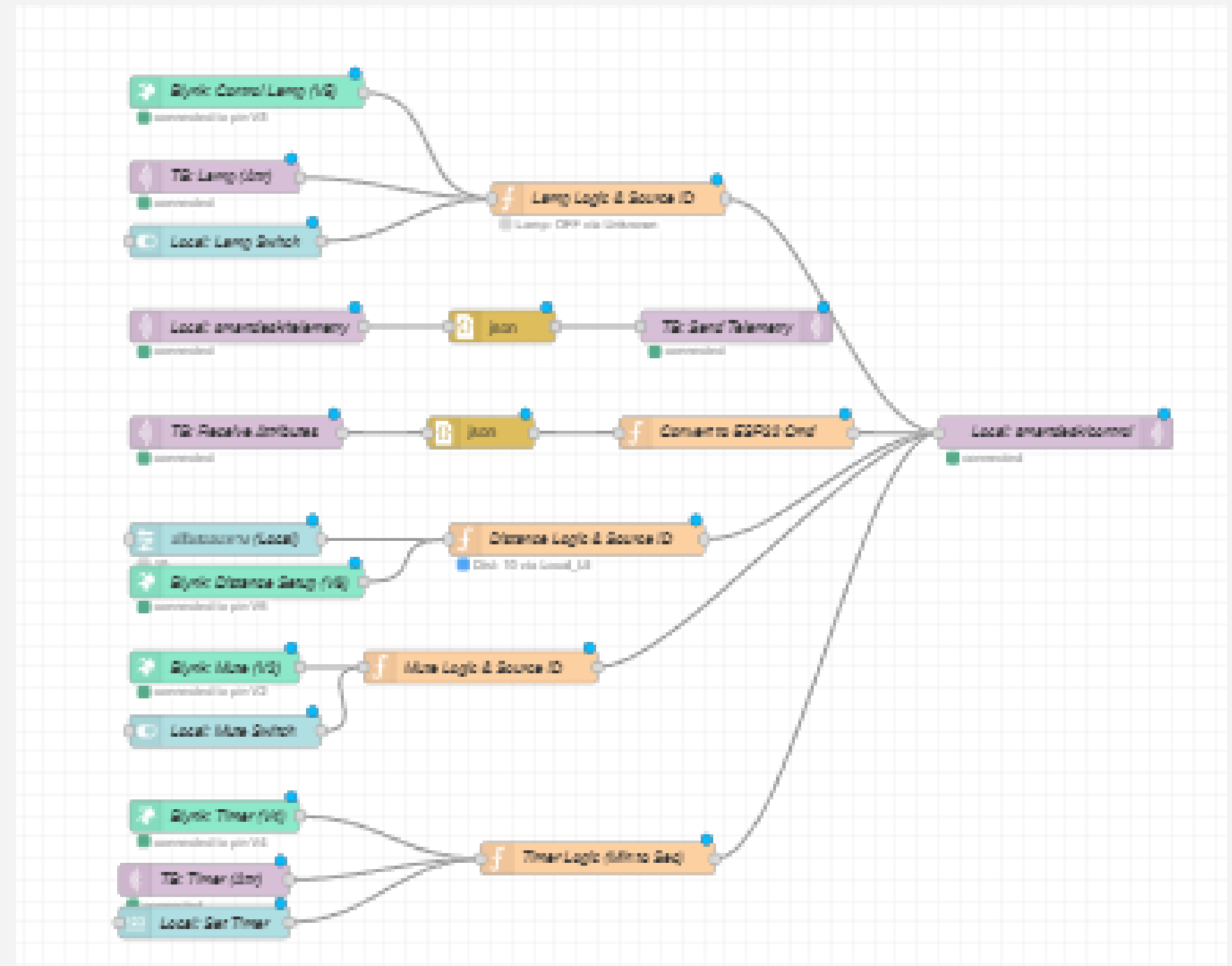
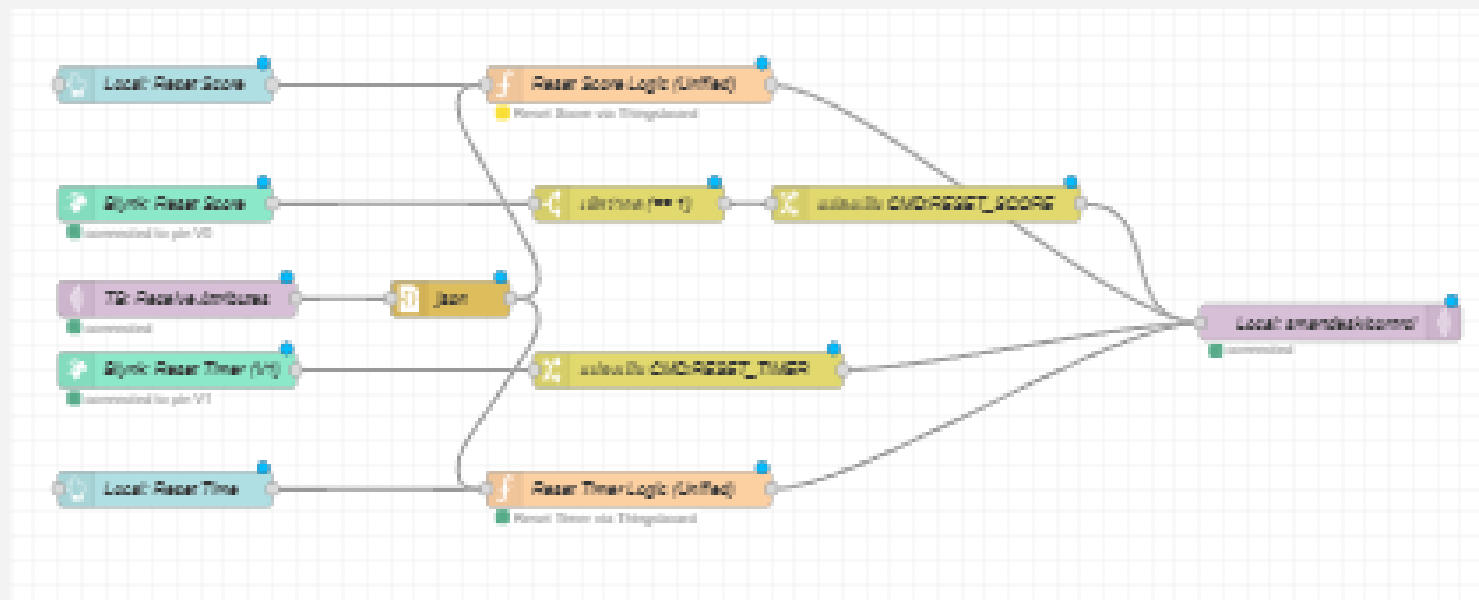
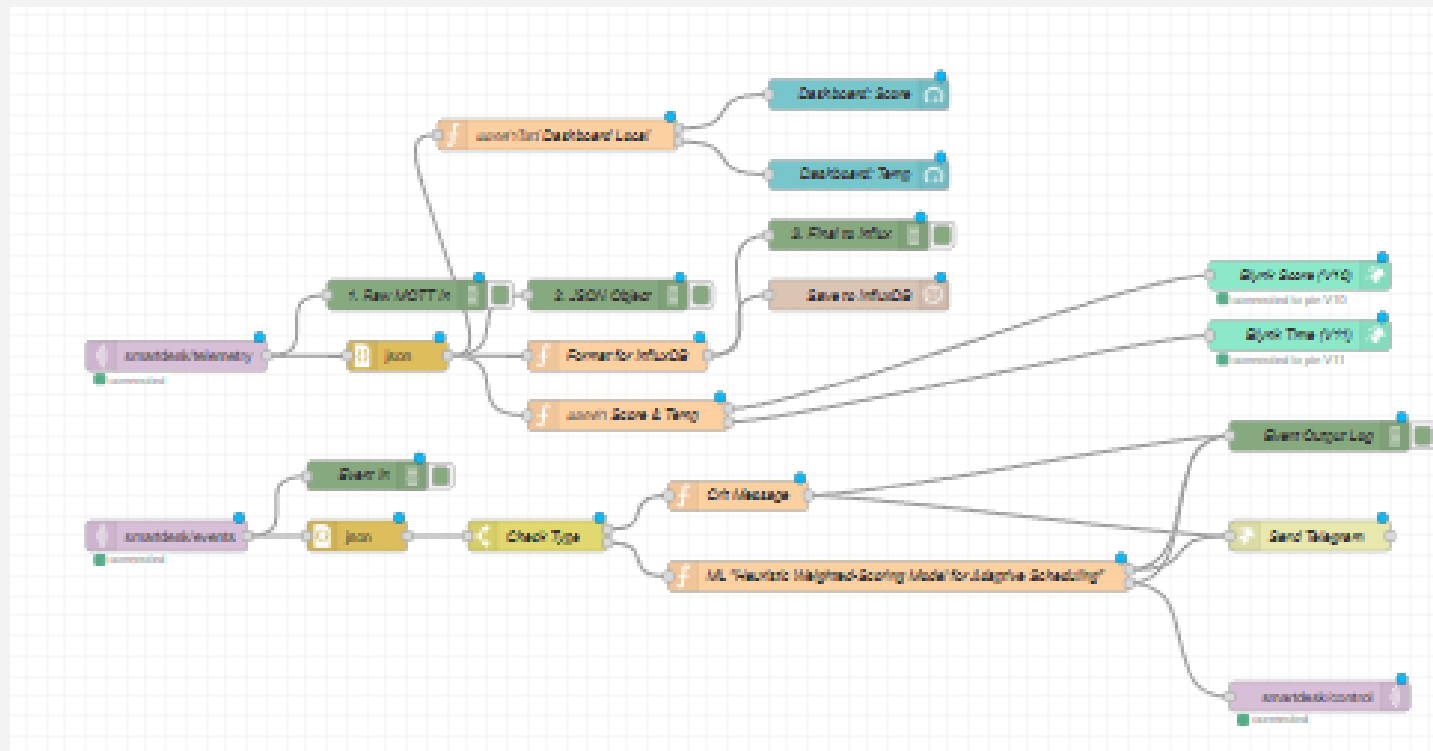
JSON Payload จาก (ESP32 Publish)

```
{"distance":40, "light":500, "temp":25.5, "score":95, "muted":0, "break_mode":0}
```

JSON Payload จาก events

- เมื่อหมดเวลาทำงาน: {"event":"session_end", "score":80, "avg_lux":450}
- เมื่อหมดเวลาพัก 5 นาที: {"event":"break_end"}
- เมื่อคะแนนต่ำกว่า 50 (วิกฤต): {"event":"critical_penalty", "score":45}

การติดตั้งและตั้งค่า Node-RED, Time Series DB



การติดตั้งและตั้งค่า Node-RED, Time Series DB



Form for saving data to InfluxDB:

- Name: Save to InfluxDB
- Server: [v2.0] IoT_Project_db
- Organization: MUICT
- Bucket: IoT_Gr19_Project
- Measurement: desk_status
- Time Precision: Milliseconds (ms)

- สร้าง Bucket จากนั้น ใช้ Token จาก Bucket ในการสร้าง server ส่งข้อมูล

Form for renaming a bucket:

Name⁺

IoT_Gr19_Project

To rename bucket use the RENAME button below

Delete Data

NEVER OLDER THAN

forever

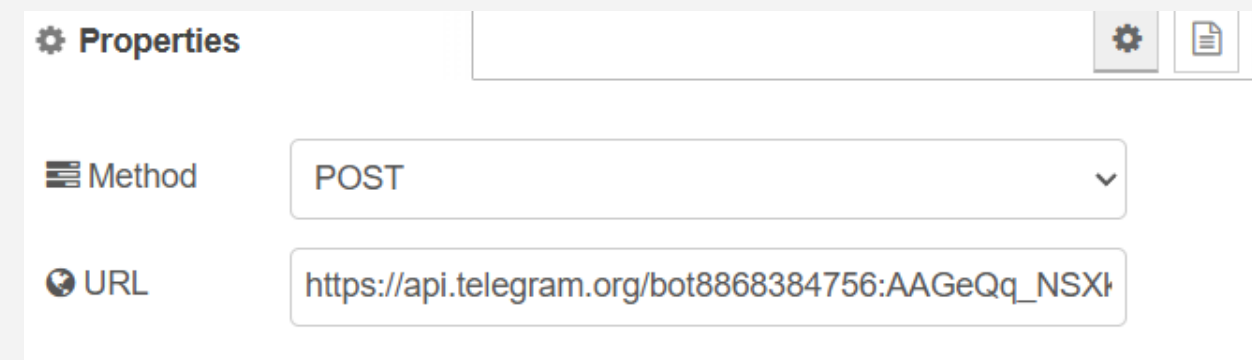
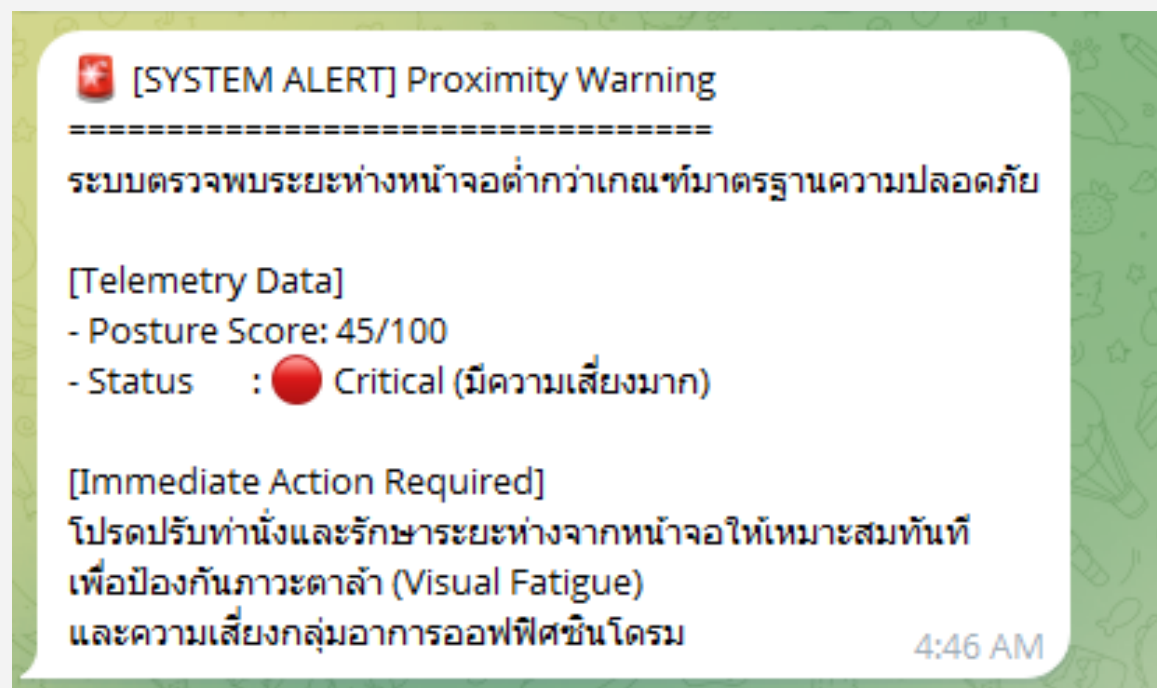
CANCEL RENAME SAVE CHANGES

การติดตั้งและตั้งค่า Node-RED, Time Series DB

Notification

Telegram

- ติดตั้งไลบรารี Telegram (node-red-contrib-telegrambot)
- สร้างบอทผ่าน BotFather ใน Telegram เพื่อเอา Token
- Logic: เมื่อ Node-RED ได้รับ Topic smartdesk/events ที่เป็น critical_penalty ให้ส่งข้อความแจ้งเตือนเข้ามือถือ



```
// --- ส่งออกไปหา Telegram ---  
msg.payload = {  
  "chat_id": "8776453068",  
  "text": alertMsg  
};  
  
return msg;
```


ML/AI

```
// --- 1. Data Ingestion ---
let score = (msg.payload.score !== undefined) ? msg.payload.score : 0;
let lux = msg.payload.light || 0;

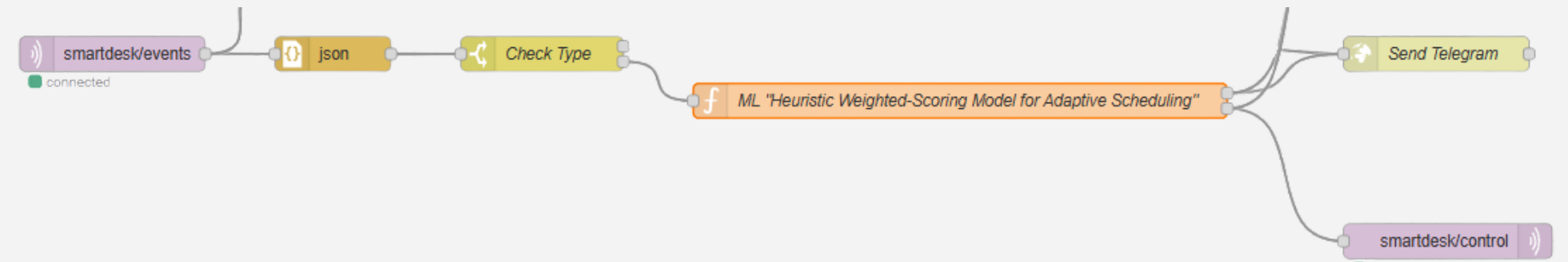
// ป้องกันค่าหลุดกรอบ (Clip values)
let cleanScore = Math.max(0, Math.min(score, 100));
let cleanLux = Math.max(0, Math.min(lux, 500));

// --- 2. Linear Regression Model ---
const intercept = 900;
const w_score = 5.4;
const w_lux = 0.72;

let nextTimer = Math.floor(intercept + (w_score * cleanScore) + (w_lux * cleanLux));

// --- 3. Status Labeling & Advisory (Professional Tone) ---
let statusLabel, advisory;

if (nextTimer >= 1620) { // >= 27 นาที
  statusLabel = "🟢 Optimal (เหมาะสม)";
  advisory = "พฤติกรรมและสภาพแวดล้อมอยู่ในเกณฑ์มาตรฐานที่ปลอดภัยต่อดวงตา";
} else if (nextTimer >= 1260) { // >= 21 นาที
  statusLabel = "🟡 Acceptable (เฝ้าระวัง)";
  advisory = "ตรวจพบระยะห่างหน้าจอที่ใกล้เกินไปในช่วง แนะนำให้ปรับระดับสายตาให้เหมาะสม";
} else {
  statusLabel = "🔴 Requires Attention (มีความเสี่ยง)";
  advisory = "สภาพแสงไม่เพียงพอ หรือระยะห่างหน้าจอไม่ปลอดภัย ระบบปรับลดเวลาทำงานเพื่อป้องกันภาวะต้อ";
}
```



Workspace Health & Ergonomics Report

[1] Assessment Summary

Status: 🔴 Requires Attention (มีความเสี่ยง)

[2] Telemetry Metrics

- Posture Score : 65/100
- Ambient Light : 0 Lux

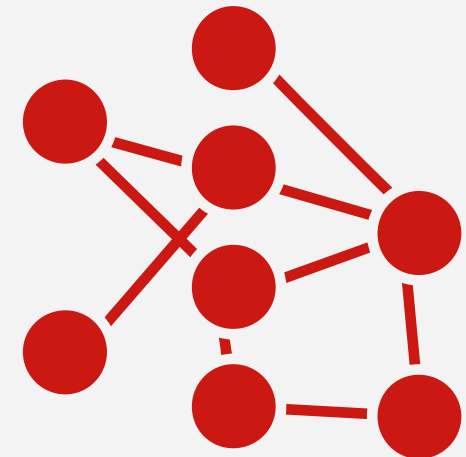
[3] System Action

- Allocated Time: 20 mins 51 secs
- Advisory : สภาพแสงไม่เพียงพอ หรือระยะห่างหน้าจอไม่ปลอดภัย ระบบปรับลดเวลาทำงานเพื่อป้องกันภาวะต้อ

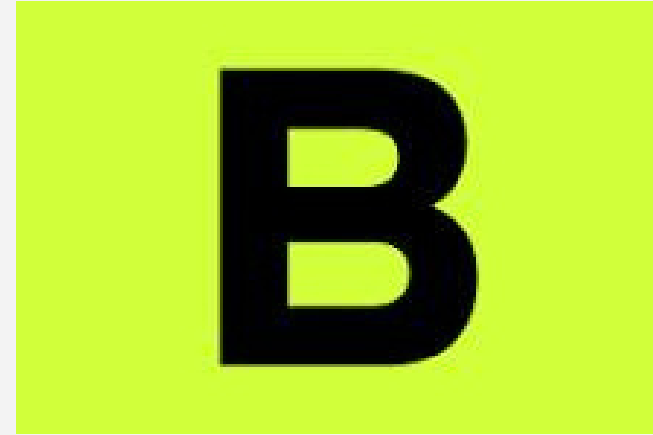
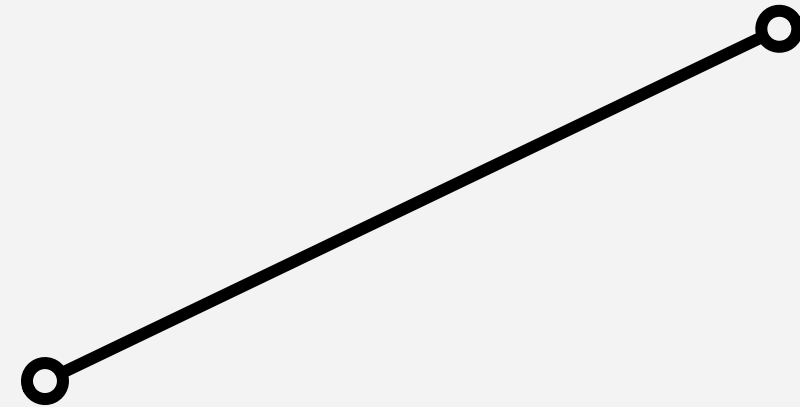
Automated by Smart Desk AI Gateway

4:13 AM

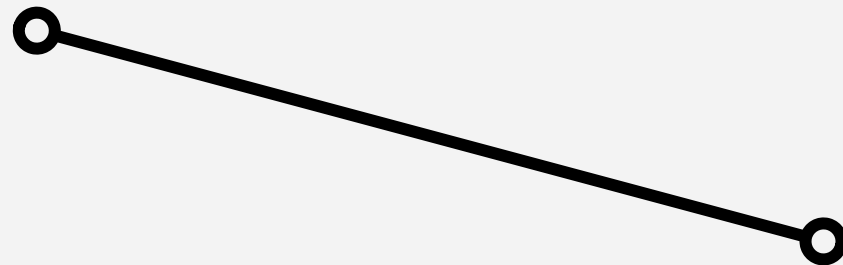
IoT Cloud



Node Red



Blynk



Thing Board

Device Data Stream

Pin port (V)

Node write event

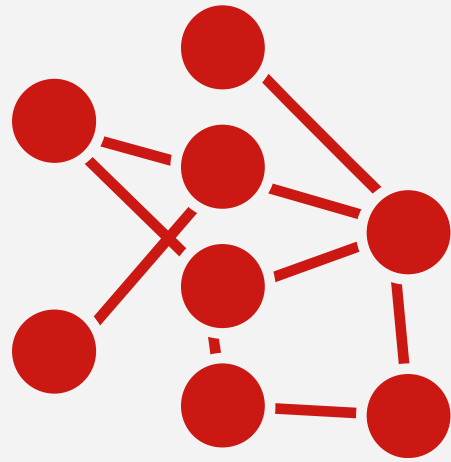
Device get/set attribute

Topic :

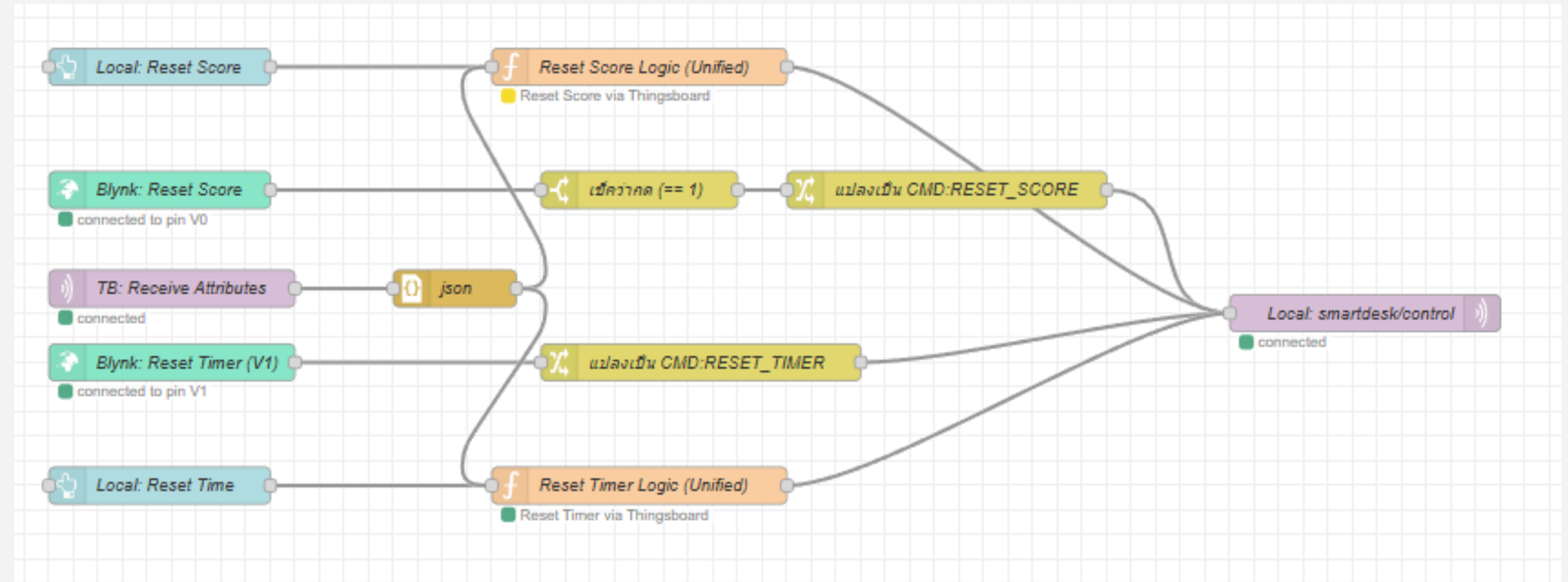
v1/devices/me/attributes

Node mqtt in/out

IoT Cloud



Node Red



IoT Cloud



Thing Board

Device get/set attribute

**Topic :
v1/devices/me/attributes**

Node mqtt in/out

Devices Device filter Include customer entities + Add device								
☐	Created time ↓	Name	Device profile	Label	State	Customer name	Groups	Is gateway
☐	2026-05-13 22:54:17	SmartDesk_Gateway	default		Active			☐  

Data

Appearance

Widget card

Actions

Layout

Target device

Device*

SmartDesk_Gateway

Initial value ×

Action

Get attribute ▼

Attribute scope

Any ▼

Attribute key*

dangerDistance ×

Action result converter

None

Function

Cancel

Apply

IoT Cloud



Blynk

Device Data Stream

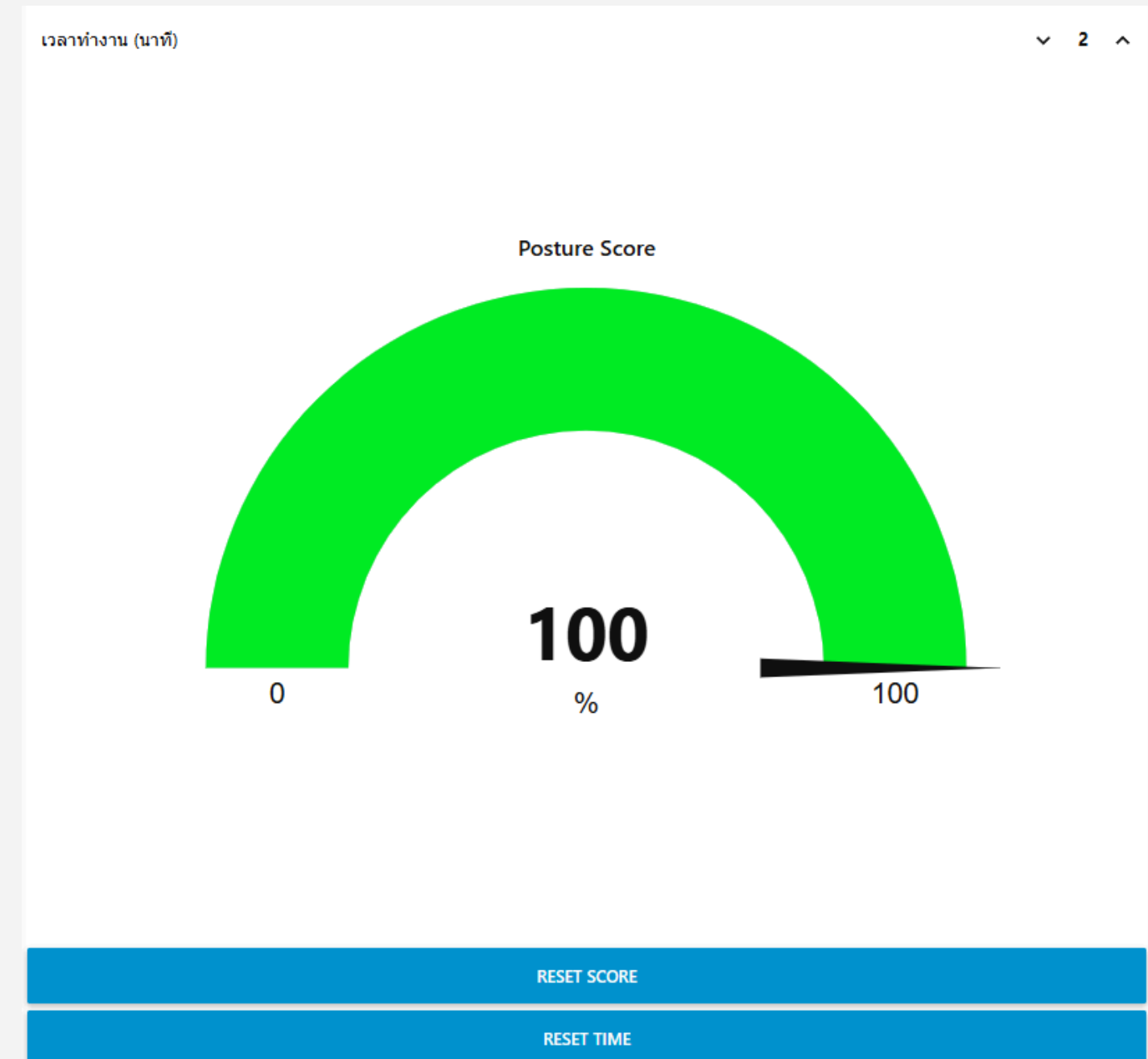
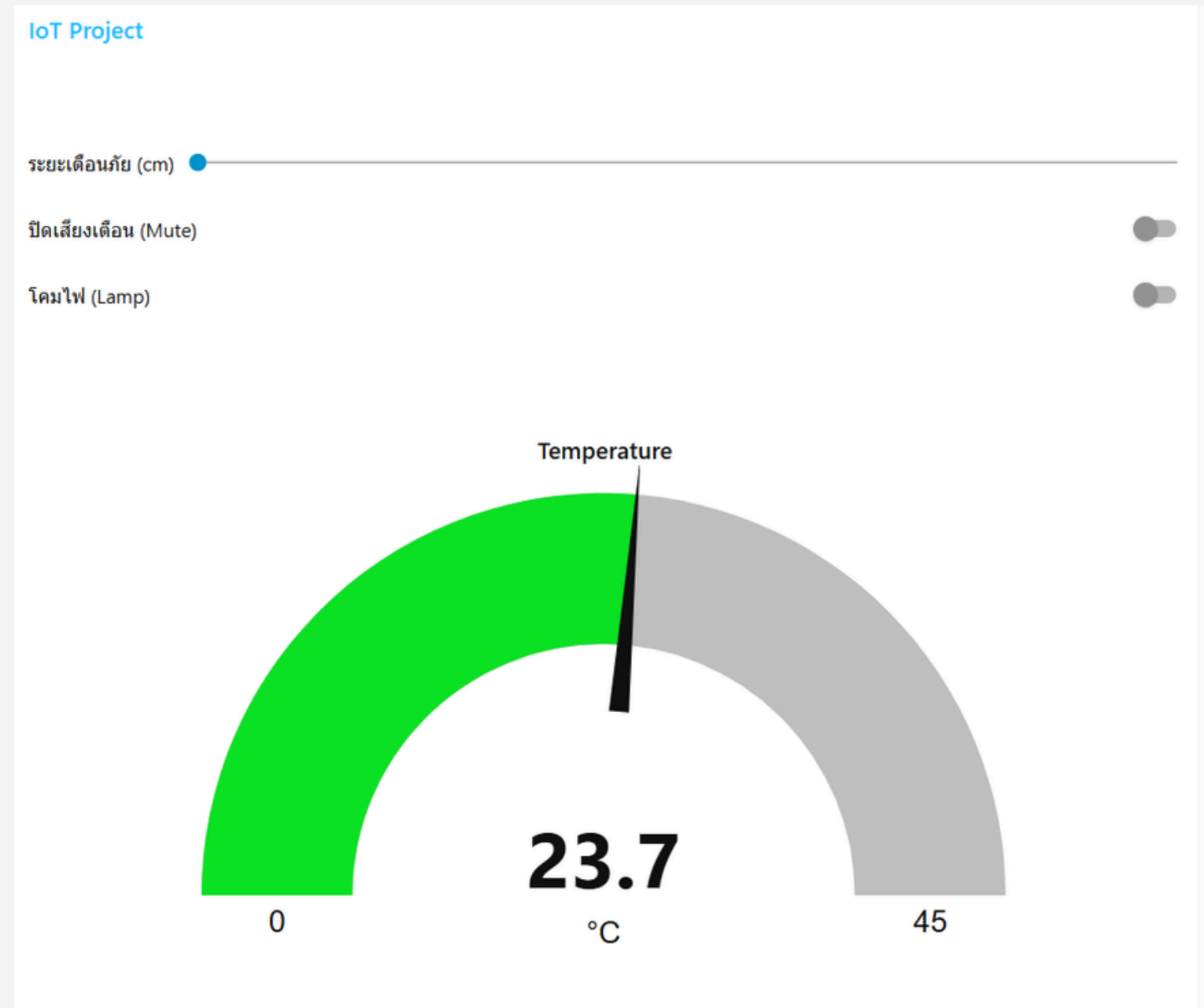
Pin port (V)

Node write event

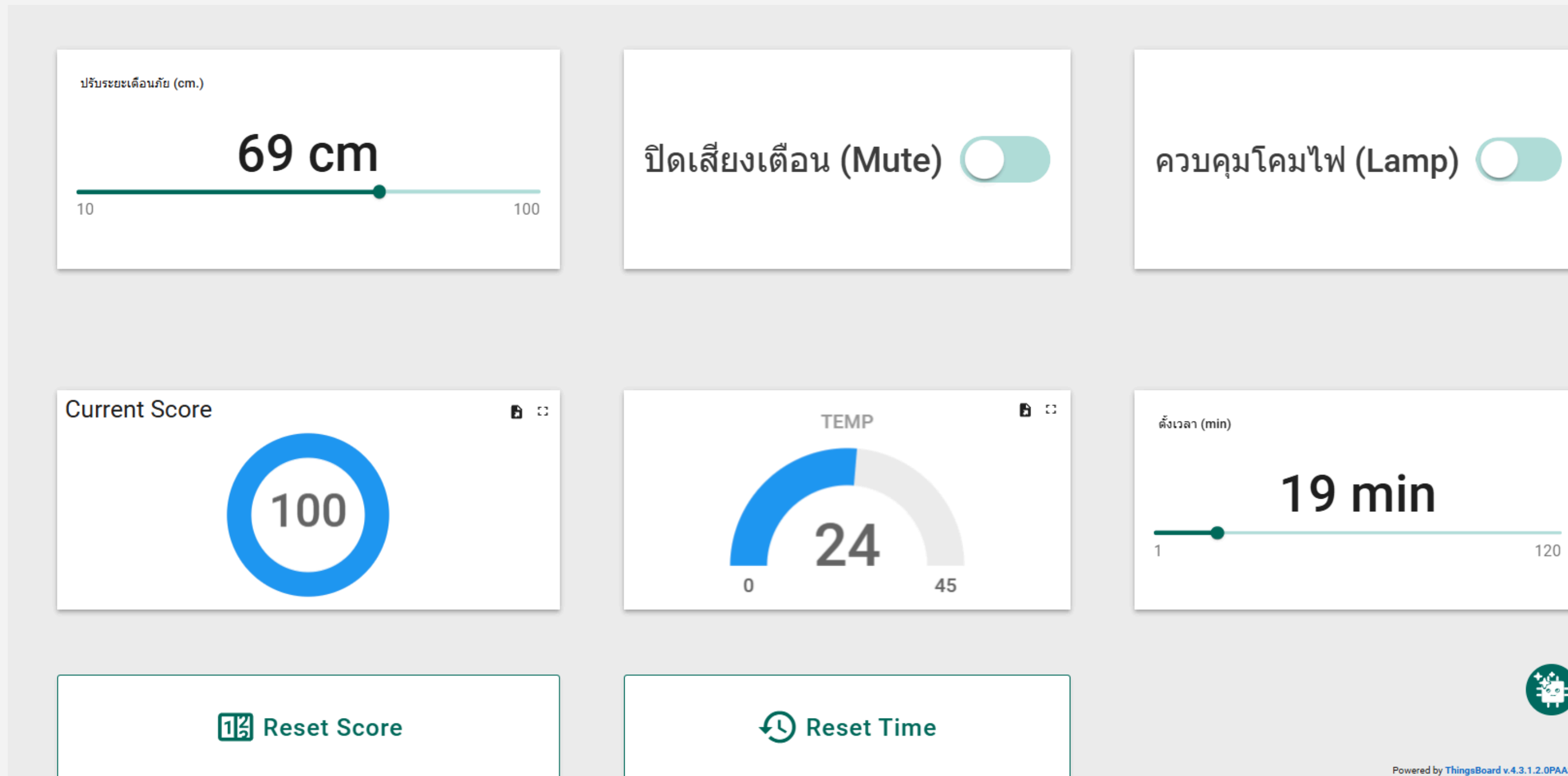
Datastreams													
Search datastream													
ID	Name	Pin	Color	Data Type	Units	Is Raw	Min	Max	Decimals	Default Value	Automation Type	% Condition	% Action
1	Reset Score	V0		Integer		false	0	1	-	0	Switch	☒	☒
2	Reset Timer	V1		Integer		false	0	1	-	0	Switch	☒	☒
3	Current Score	V10		Integer		false	0	100	-	0	Switch	☒	☒
4	Remaining Time	V11		Integer		false	0	45	-	0	Switch	☒	☒
5	Set Time	V4		Integer		false	0	120	-	0	Switch	☒	☒
6	Control Lamp	V5		Integer		false	0	1	-	0	Switch	☒	☒
7	Distance Setup	V6		Integer		false	1	100	-	0	Switch	☒	☒
8	Mute	V2		Integer		false	0	1	-	0	Switch	☒	☒

☐	Name	Auth Token	Device Owner	Status
	Smart Desk Controller	kaUB1iVWvitMaVf9UI22btgbabqA...	jaffreyamaga.phil@student.mahid...	Online

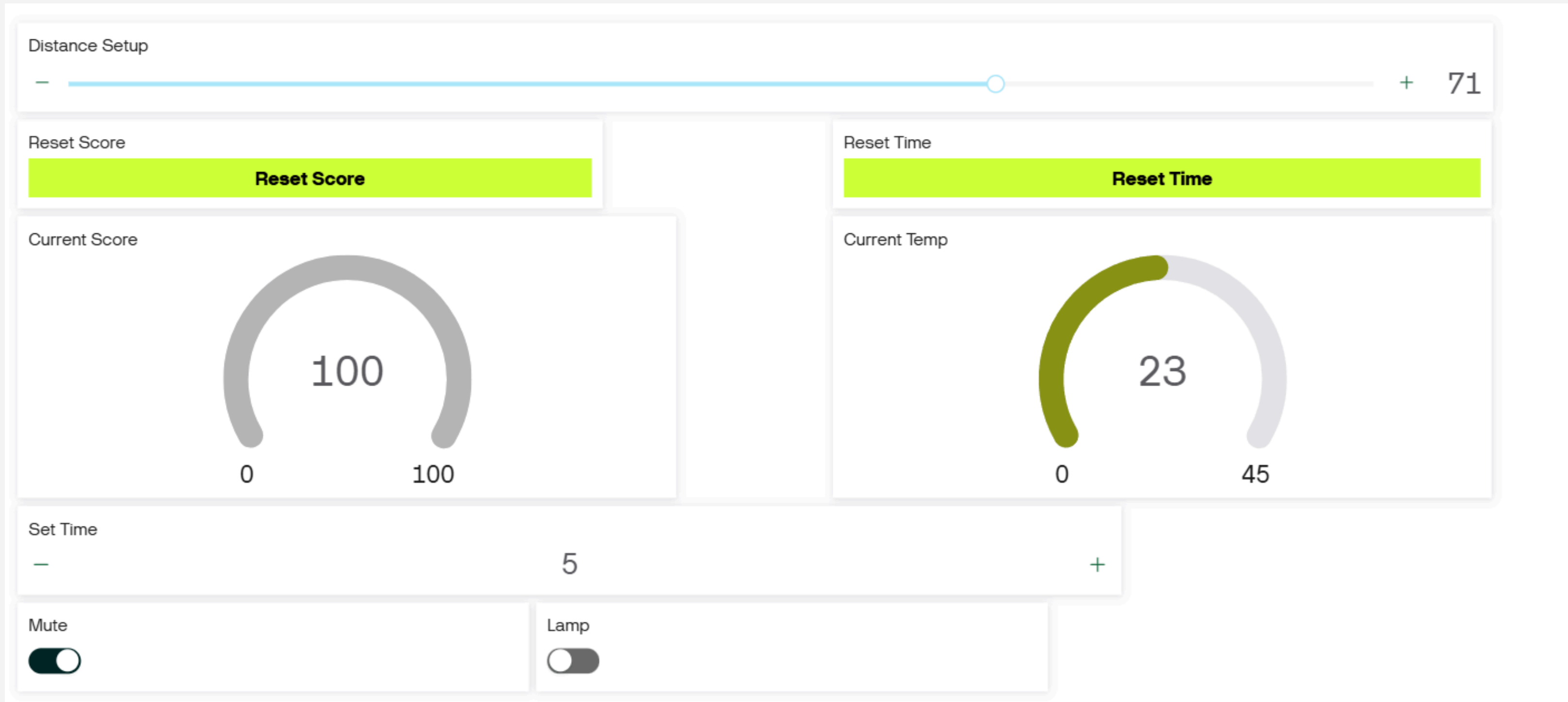
Local & Remote Dashboard



Local & Remote Dashboard



Local & Remote Dashboard



ปัญหาและแนวทางแก้ไข

1. ปัญหา: ความขัดแย้งในการสั่งงานจากหลายแพลตฟอร์ม

- **แนวทางแก้ไข:** ออกแบบสถาปัตยกรรมแบบ Edge Gateway โดยใช้ Node-RED เป็นสมองกลศูนย์กลาง รับคำสั่งจากทุกแพลตฟอร์ม จัดระเบียบแยกแยะที่มา แล้วแปลงเป็นคำสั่งรูปแบบเดียว

2. ปัญหา: การส่งคำสั่งซ้ำจาก Thingsboard ไม่ทำงาน

- **แนวทางแก้ไข:** ปรับเปลี่ยน UI และ Logic โดยเปลี่ยนไปใช้ Switch Control หรือเขียนฟังก์ชัน Toggle แทน เพื่อให้มีการส่งค่า 1 ทำงาน สลับกับ 0 เคลียร์ค่า ทำให้ Node-RED ตรวจจบการเปลี่ยนแปลงได้ทุกครั้งที่เกิดสั่งงาน

3. ปัญหา: ความเครียดจากการถูกจับผิดตลอดเวลา

- **แนวทางแก้ไข:** ประยุกต์ใช้หลักการ Pomodoro Technique โดยเขียน State Machine ใน ESP32 แทรก "โหมดพักสายตา 5 นาที " เมื่อหมดรอบทำงาน ระบบจะหยุดประเมินทำนองชั่วคราว เปลี่ยนไฟ RGB เป็นสีฟ้าให้ดูผ่อนคลาย และทำงานร่วมกับ Edge ML เพื่อดึงเวลาทำงานรอบใหม่มาตั้งค่าโดยอัตโนมัติเมื่อหมดเวลาพัก

ประโยชน์ และแนวทางที่นำไปประยุกต์ใช้

1. ประโยชน์ของโครงการ

- **ป้องกันปัญหาสุขภาพในระยะยาว:** ช่วยลดความเสี่ยงการเกิด Office Syndrome และ อาการตาล้า สายตาสั้นเทียม
- **เพิ่มประสิทธิภาพการทำงาน:** ระบบ Dynamic Timing ช่วยหาจุดสมดุลระหว่างการทำงานและการพัก

2. แนวทางการนำไปประยุกต์ใช้

- **Smart Home & Office โต๊ะทำงานอัจฉริยะ: Work From Home:** พัฒนาเป็นอุปกรณ์เสริมสำเร็จรูป สำหรับพนักงานที่ทำงานที่บ้าน ซึ่งไม่มีหัวหน้าหรือเพื่อนร่วมงานคอยเตือนเรื่องบุคลิกภาพ
- **Healthcare เครื่องมือช่วยเหลือนักกายภาพบำบัด: Posture Correction:** ประยุกต์ใช้เซ็นเซอร์หลายจุดเพื่อตรวจจับท่าทางการนั่งที่ละเอียดขึ้น เพื่อส่งข้อมูลให้นักกายภาพบำบัดประเมินอาการและออกแบบท่าบริหารที่ตรงจุดสำหรับคนไข้แต่ละราย

AI Assistance



- **Design**



- **Debug**

สมาชิกผู้จัดทำ



Jeffrey Phillips รหัสนักศึกษา 6787094



Jittipat Paenyoi รหัสนักศึกษา 6787098